



UNIVERSIDAD  
**NACIONAL**  
DE COLOMBIA

Universidad Nacional de Colombia - Sede Bogotá  
Facultad de Ingeniería  
Departamento de Sistemas e Industrial  
Curso: Ingeniería de Software 1 (2016701)

### Informe Testing Unitario

Nicolás Aguirre Velásquez – [niaguirrev@unal.edu.co](mailto:niaguirrev@unal.edu.co)

Diego Alexander Ibañez Torres – [dibanezt@unal.edu.co](mailto:dibanezt@unal.edu.co)

Gabriel Esteban Huertas Ruiz – [ghuertas@unal.edu.co](mailto:ghuertas@unal.edu.co)

Jhon Freddy Patiño Meza – [jhpatinom@unal.edu.co](mailto:jhpatinom@unal.edu.co)



### Herramientas utilizadas:

Para las pruebas unitarias empleamos

- JUnit
- Mockito: para la simulación de dependencias

### Funcionalidad 1: Autenticación de usuarios

**RUTA:** project-backend/src/test/java/com/parking/backend/service/AuthServiceTest

### Test #1: Inicio de sesión con credenciales válidas

**Método:** loginWithCorrectCredentialsReturnsAuthResponse

Se verifica que el sistema autentique correctamente a un usuario con credenciales válidas y genere un token JWT.

### Resultados esperados:

- Se genera un token JWT.

- Se retorna la información del usuario autenticado.
- El contador de intentos fallidos se reinicia a cero.

## Test #2: Bloqueo de cuenta tras cinco intentos fallidos

**Método:** loginAfter5FailedAttemptsBlocksAccount

Se verifica que el sistema bloquee una cuenta al realizar 5 intentos fallidos consecutivos de autenticación. Se valida también que el contador de intentos fallidos se actualice correctamente

Caso límite validado: Transición exacta entre cuatro y cinco intentos fallidos

### Resultados esperados:

- Se genera una excepción de credenciales inválidas.
- La cuenta queda bloqueada.
- El contador de intentos fallidos aumenta a cinco.

## Test #3: Registro con nombre de usuario duplicado

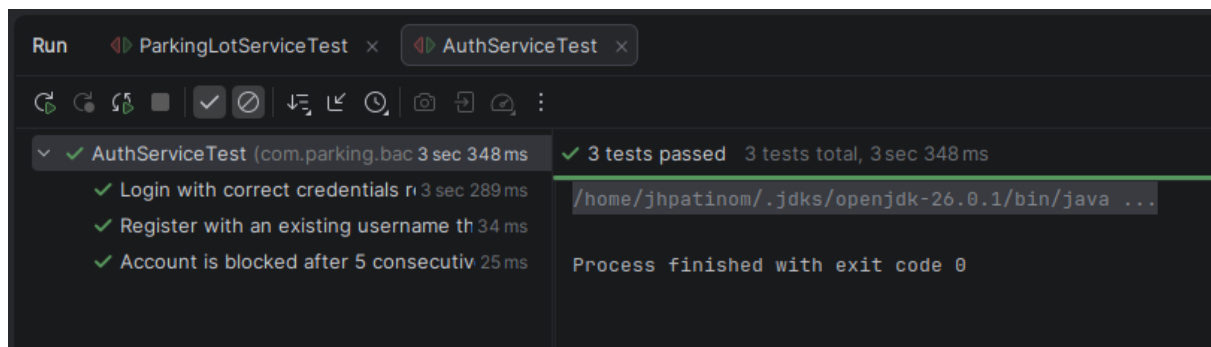
**Método:** registerWithDuplicateUsernameThrowsException

Se verifica que el sistema impida el registro de usuarios con nombres de usuario ya existentes

Caso límite validado: Registro utilizando un identificador ya existente

### Resultados esperados:

- Se genera una excepción.
- El usuario no es almacenado.



```

Run    ParkingLotServiceTest x    AuthServiceTest x
[Icons]
v ✓ AuthServiceTest (com.parking.bac 3 sec 348ms) ✓ 3 tests passed 3 tests total, 3 sec 348 ms
  ✓ Login with correct credentials ri 3 sec 289 ms
  ✓ Register with an existing username th 34 ms
  ✓ Account is blocked after 5 consecutiv 25 ms
/home/jhpatinom/.jdk/openjdk-26.0.1/bin/java ...
Process finished with exit code 0

```

## **Funcionalidad 2: Gestión de Parqueaderos**

**ruta:** project-backend/src/test/java/com/parking/backend/service/ParkingLotService

### **Test #4: Validación de horario inválido**

**Método:** createThrowsWhenOpeningAfterClosing

Se verifica que el sistema rechace configuraciones donde la hora de apertura sea posterior a la hora.

Caso límite validado: Horarios inconsistentes que violan las reglas de negocio.

#### **Resultados esperados:**

- Se genera una excepción.
- El parqueadero no se registra.

### **Test #5: Eliminación de parqueadero ya existente**

Se verifica que un parqueadero pueda eliminarse correctamente cuando ya existe en el sistema.

**Método:** deleteSucceedsWhenLotExists

#### **Resultados esperados:**

- La operación se ejecuta sin errores.
- Se invoca la eliminación correspondiente.

### **Test #6: Consulta de parqueaderos registrados**

Se verifica que el servicio retorne la lista completa de parqueaderos almacenados.

**Método:** getAllReturnsRepositoryList

#### **Resultados esperados:**

- Se retorna una lista con exactamente dos elementos.
- Se consulta correctamente el repositorio.

